Documentation du déploiement de l'application Vite & Gourmand

Voici la démarche ainsi que les différentes étapes du déploiement de mon application :

Sommaire

1. Objectif du déploiement..... 2

2. Architecture technique..... 2

3. Préparation du projet au déploiement..... 3

4. Création de l'application sur Heroku..... 4

5. Configuration du projet pour Heroku..... 5

6. Déploiement de l'application..... 5

7. Mise à jour de l'application..... 6

8. Processus de déploiement via Git..... 6

9. Vérification du déploiement..... 7

a) Logs Heroku heroku logs --tail..... 7

b) Test de l'application..... 7

10. Synchronisation des données analytiques..... 7

Conclusion..... 7

1. Objectif du déploiement

L’objectif du déploiement est de rendre l’application Vite-Gourmand accessible en ligne afin que les utilisateurs puissent consulter les menus, passer des commandes et suivre leur état.

Le déploiement doit répondre aux critères suivants :

- Application accessible en ligne
- Déploiement reproductible
- Mise à jour rapide après modification du code
- Utilisation d’un système de versionnement (Git)

Pour cela, l’application est déployée sur Heroku et versionnée via GitHub.

2. Architecture technique

L’application repose sur les technologies suivantes :

Backend :

- PHP
- Architecture MVC (Model (base de données) – View – Controller)
- Composer pour l’autoload et les dépendances (PHPMailer, MongoDB, PHPUnit)

Base de données :

- Relationnelle : MySQL (données principales : utilisateurs, commandes, menus, etc…)
- Non relationnelle (NoSQL) : MongoDB Atlas (statistiques des commandes)

Outils de déploiement :

- Git
- GitHub
- Heroku
- Heroku CLI
- Docker et XAMPP (environnements locaux)

3. Préparation du projet au déploiement

Avant de déployer l'application, plusieurs éléments ont été configurés pour rendre le projet compatible avec Heroku.

Initialisation Git :

Le projet a été versionné avec Git via le terminal :

git init

git add .
git commit -m "Initial commit – projet complet"

Le projet a également été relié au dépôt distant GitHub via le terminal :

git remote add origin https://github.com/Mathieupalle/vite-gourmand.git

git push -u origin develop

Cela permet de sauvegarder le code et de suivre les modifications.

Ensuite, chaque fonctionnalité ou correction est développée sur une branche créée à partir de « develop ». Elle est fusionnée dans « develop » par une pull request sur GitHub, puis « develop » est fusionnée dans « main » de la même façon. Je récupère ensuite la branche « main » en local :

git checkout -b fix/ma-correction develop

git push -u origin fix/ma-correction

git checkout main

git pull origin main
La pull request est créée et fusionnée depuis le site GitHub.

4. Création de l’application sur Heroku

Heroku est utilisé comme plateforme d’hébergement en ligne.

Connexion au compte Heroku via le terminal :

heroku login

Création de l’application via le terminal :

heroku create vite-gourmand-mathieu

Cette commande crée :

- Une application Heroku
- Une URL publique
- Un dépôt Git distant Heroku

Voici l’URL générée :
https://vite-gourmand-mathieu-db469fca4eaf.herokuapp.com/

5. Configuration du projet pour Heroku

a) Ajout du remote Heroku

Pour connecter le projet local à Heroku :

heroku git:remote -a vite-gourmand-mathieu

Vérification :

git remote -v

b) Ajout des variables d'environnement

Les informations sensibles ne sont pas stockées dans le code. Elles sont configurées dans les variables d'environnement Heroku (Settings > Config Vars, ou heroku config:set dans le terminal) :

- **BASE_URL** : racine des URL du site
- **JAWSDB_URL** : connexion MySQL, ajoutée automatiquement par l'add-on JawsDB (mysql://utilisateur:********@hôte:3306/base)
- **MONGODB_URI** : connexion MongoDB Atlas (mongodb+srv://utilisateur:********@cluster0…mongodb.net/)
- **APP_URL** : adresse complète du site, utilisée dans les liens des emails
- **SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM** : envoi des emails avec PHPMailer
- **CONTACT_TO** : adresse qui reçoit les messages du formulaire de contact

Cela permet de :

- Sécuriser les identifiants
- Séparer la configuration et le code
- Ne jamais publier de mot de passe sur GitHub

En local, ces valeurs sont dans le fichier config.local.php, ignoré par Git.

6. Déploiement de l'application

Le déploiement se fait via Git, à partir de la branche « main » à jour. Commande utilisée via le terminal :
git push heroku main

Heroku effectue alors automatiquement :

1. Récupération du code
2. Installation des dépendances (composer install, buildpack PHP)
3. Démarrage du serveur Apache défini dans le Procfile (dossier public/)
4. Mise en ligne

Une fois terminé, l'application est accessible via l'URL publique.

7. Mise à jour de l'application

Lorsqu'une modification est effectuée dans le code :

1. Modification locale sur une branche dédiée
2. Commit Git, puis pull request vers « develop », puis vers « main »
3. Push de « main » vers Heroku

Commandes utilisées via le terminal :
git checkout main
git pull origin main
git push heroku main

Heroku reconstruit alors l'application et la relance.

8. Processus de déploiement via Git

Le projet utilise Git et GitHub à chaque étape. Voici le processus :

1. Développement en local (XAMPP ou Docker) et tests PHPUnit
2. Commit du code sur une branche (Git)
3. Pull request sur GitHub vers « develop », puis vers « main »
4. Push de la branche « main » vers Heroku (git push heroku main)
5. Reconstruction et redémarrage de l'application par Heroku
6. Application en ligne

Ce système permet :

- Un déploiement rapide
- Une traçabilité des modifications
- Une restauration facile en cas d'erreur

9. Vérification du déploiement

Après chaque déploiement, plusieurs vérifications sont effectuées :

a) Logs Heroku

heroku logs --tail

Cela permet de détecter les erreurs.

b) Test de l'application

Vérification des fonctionnalités principales :

- Affichage des menus
- Création de commandes
- Gestion des utilisateurs
- Affichage des statistiques
- Tests automatisés lancés en local avant le déploiement (vendor/bin/phpunit)

10. Synchronisation des données analytiques

L'application utilise également MongoDB Atlas pour les statistiques. Lors de la création ou de la modification d'une commande :

1. La commande est enregistrée dans MySQL
2. La commande est recopiée (ou mise à jour) dans MongoDB Atlas par la classe MigrateOrders.php.

Pour recopier toutes les commandes existantes dans MongoDB (par exemple après un import), un script est lancé sur Heroku :
heroku run php scripts/migrateOrders.php -a vite-gourmand-mathieu

Cela permet d'alimenter :

- Le nombre de commandes par menu (graphique)
- Le chiffre d'affaires par menu
- Les filtres par menu et par période

Conclusion :

Le déploiement de l'application Vite-Gourmand repose sur :

- Git et GitHub pour le versionnement (branches et pull requests)
- Heroku pour l'hébergement
- MongoDB Atlas pour les statistiques

Cette architecture permet :

- Un déploiement rapide
- Des mises à jour simples
- Une application accessible en ligne
- Une gestion sécurisée de la configuration

Le système de déploiement mis en place permet de maintenir facilement l’application et d’ajouter de nouvelles fonctionnalités.